Operating Systems Laboratory

Part 1: Operating System Fundamentals and Process Management

Q1: Operating System Fundamentals

Define an operating system and explain the roles of the kernel and shell. Describe two main functions performed by an operating system.

Answer:

An operating system is a program that acts as an intermediary between a user of a computer and the computer hardware.

The role of the kernel is that it is the one program running at all times on the computer, and it provides core services beneath the system-call interface like the file system, CPU scheduling, and memory management.

The role of the shell is to work as a command-line interface (CLI) or command interpreter that allows direct command entry, where it primarily fetches a command from the user and executes it.

2 main functions performed by an operating system are acting as a resource allocator to manage all resources and decide between conflicting requests for efficient and fair use, and acting as a control program that controls the execution of programs to prevent errors and improper use of the computer.

Q2: Virtualization and Operating System Concepts

Explain the concept of virtualization. Differentiate between a host operating system and a guest operating system, and explain the purpose of Oracle VirtualBox.

Answer:

Virtualization is a computing environment concept that allows operating systems to run applications within other OS, where the OS is natively compiled for the CPU and runs guest OS that is also natively compiled.

The diffrence between them is that the host operating system is the native main OS running directly on the physical hardware, while the guest operating system is the OS running inside a virtual machine on top of the host. The purpose of Oracle VirtualBox is to act as a virtual machine manager (VMM) that provides virtualization services, allowing users to safely install

and run multiple guest OSes like Ubuntu Linux for testing, or other stuff without affecting the host machine.

Q3: Linux Commands

Write and execute the Linux commands required to perform the following operations:

a. Display the current working directory.

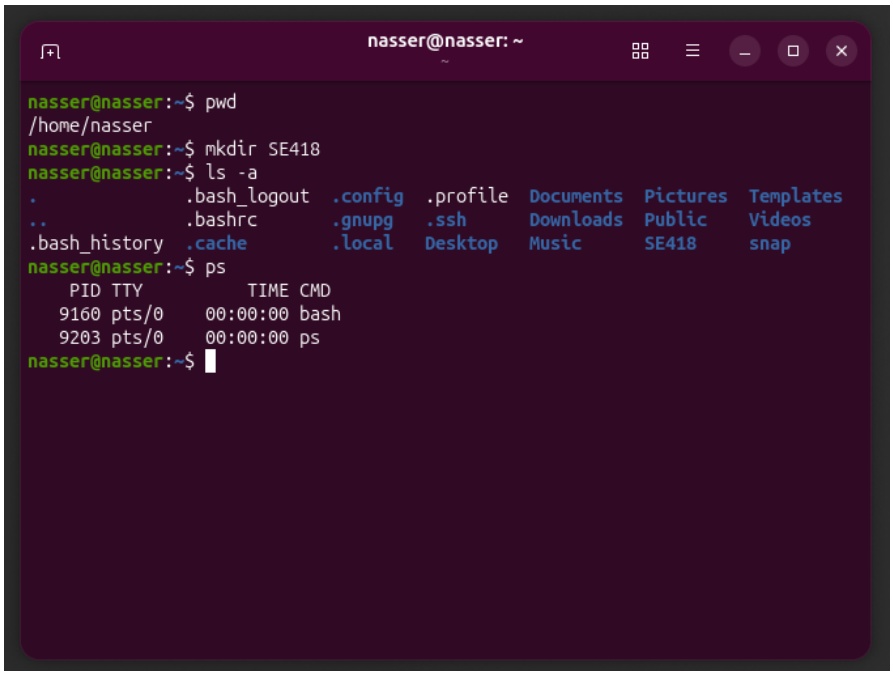
b. Create a new directory named SE418.

c. List all files, including hidden files.

d. Display currently running processes.

Include screenshots showing the executed commands and their outputs.

Answer:



Q4: Process Management Using C

Write a C program that creates a child process using the fork() system call. The program must:

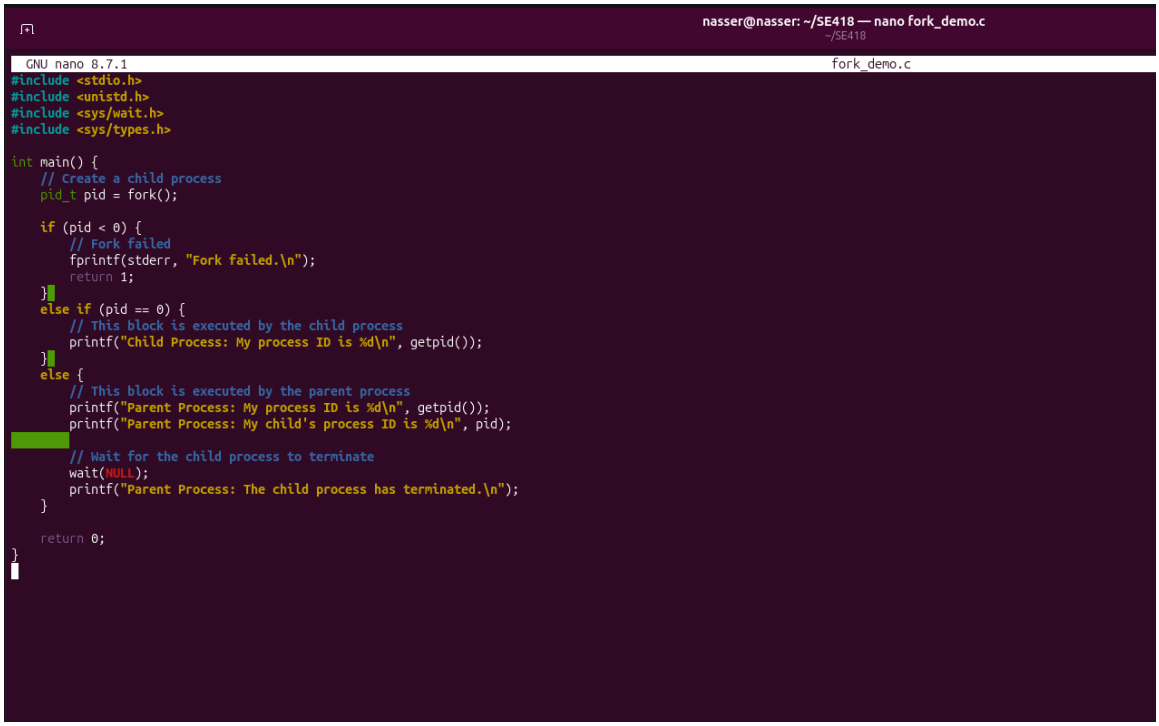
a. Display the process ID of the parent process **13667**.

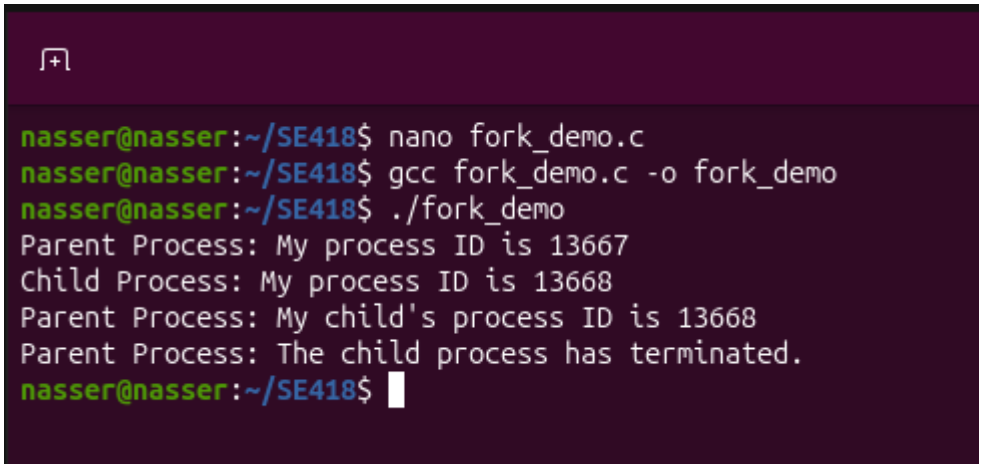
b. Display the process ID of the child process **13668**.

c. Use wait() to allow the parent process to wait for the child to terminate.

Compile and execute the program in Ubuntu and provide a screenshot of the output.

Answer (Source Code):





Q5: Operating System Security

Explain two cybersecurity issues related to operating systems, such as unauthorized access and privilege escalation. Describe how Linux file permissions and user privileges help protect system resources.

Answer:
2 main security issues in operating systems are:

1- invalid access attempts (unauthorized access) from outsiders trying to break user authentication.
2- privilege escalation, which happens when a user or attacker manages to change to an effective ID with more rights than they should have.

Linux file permissions and user privileges help protect system resources by providing a protection mechanism for controlling the access of processes or users to resources defined by the OS. The system first distinguish among users by assigning User identities (user IDs) and Group identifiers (group IDs). These IDs are associated with all files and processes of that user to determine access control and permission management (using protection system calls like chmod, umask, and chown), which ensures that all access to system resources is controlled and concurrent proceses do not interfere with each other.

Part 2: Operating System Algorithms and Resource Management

Q6: CPU Scheduling Algorithms

Consider the following processes:

P1 (Arrival: 0, Burst: 5), P2 (Arrival: 1, Burst: 3), P3 (Arrival: 2, Burst: 2)

Apply FCFS and SJF (Non-preemptive). Draw the Gantt chart for each, calculate average waiting time, and compare.

Answer:

FCFS:

P1 waiting time = 0 - 0 = 0

P2 waiting time = 5 - 1 = 4

P3 waiting time = 8 - 2 = 6

Average waiting time = (0 + 4 + 6) / 3 = 10 / 3 = 3.33

SFJ:

P1 waiting time = 0 - 0 = 0

P3 waiting time = 5 - 2 = 3

P2 waiting time = 7 - 1 = 6

Average waiting time = (0 + 3 + 6) / 3 = 9 / 3 = 3

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	2

FCFS

Gantt Chart Timeline:

0	1	2	3	4	5	6	7	8	9	10
P1					P2			P3		

FCFS Process Table

Process	Arrival Time	Burst Time	Start Time	Completion Time	Turnaround Time	Waiting Time
P1	0	5	0	5	5	0
P2	1	3	5	8	7	4
P3	2	2	8	10	8	6
Average Waiting Time (FCFS)						3.333333333

SJF

Gantt Chart Timeline:

0	1	2	3	4	5	6	7	8	9	10
P1					P3			P2		

SJF Process Table

Process	Arrival Time	Burst Time	Start Time	Completion Time	Turnaround Time	Waiting Time
P1	0	5	0	5	5	0
P3	2	2	5	7	5	3
P2	1	3	7	10	9	6
Average Waiting Time (SJF)						3

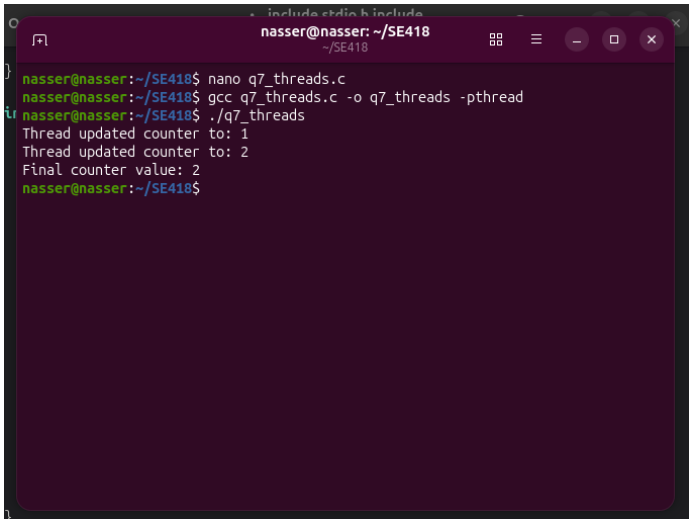
Q7: Multithreading and Process Synchronization

Explain the concept of a race condition in multithreaded programs. Write a C program using POSIX Threads (pthread) to create two threads that update a shared counter. Use a mutex to protect the shared variable and explain how synchronization prevents race conditions.

Answer (Source Code & Explanation):

Race condition occurs when concurrent processes or threads access the same shared data at the same time and the final outcome dependes on the access order. Because operations like incrementing a counter are multi step machine language operations, threads can be interrupted at any time, leaving execution partially complete and making the shared data inconsistent. Process syncronization prevents race conditions by using software tools like mutex locks to protect the critical section. A thread must acquire() the lock before entering its critical section to modify the shared variable, and release() the lock afterward. This satisfies mutual exclusion, which guarantees that while one thread is executing in its critical section, no other thread can be executing in its own critical section.





Q8: Deadlock Avoidance Using Banker's Algorithm

Explain Banker's Algorithm. Given processes P0, P1, P2 with Allocations and Max needs, and Available (1,1,1):

a. Calculate the Need matrix.

b. Determine whether the system is in a safe state.

c. Identify a safe sequence if one exists.

Answer:

The purpose of the Banker's Algorithm is in deadlock avoidance. It checks the current resource allocations against the maximum needs of all processes to determine if the system is in a "safe state" before granting any resource request, so the operating system avoids entering a deadlock.

a) Need = Max - Allocation:

P0 Need = (2 - 0, 2 - 1, 1 - 0) = (2, 1, 1)
P1 Need = (3 - 1, 1 - 0, 2 - 1) = (2, 1, 1)
P2 Need = (2 - 1, 2 - 1, 1 - 0) = (1, 1, 1)

b) Yes, it is in a safe state

c) P2, P0, P1

Q9: Memory Allocation Algorithms

Memory Blocks (KB): 100, 500, 200, 300, 600

Process Sizes (KB): 212, 417, 112, 426

Apply First Fit, Best Fit, and Worst Fit. Show allocation results and compare utilization. Assume each allocated process reduces remaining block size.

Answer:

First Fit:

Process	Size (KB)	Allocated Block	Remaining Size
P1	212	B2 (500)	288
P2	417	B5 (600)	183
P3	112	B2 (288)	176
P4	426	Not Allocated	-

Best Fit:

Process	Size (KB)	Allocated Block	Remaining Size
P1	212	B4 (300)	88
P2	417	B2 (500)	83
P3	112	B3 (200)	88
P4	426	B5 (600)	174

Worst Fit:

Process	Size (KB)	Allocated Block	Remaining Size
P1	212	B5 (600)	388
P2	417	B2 (500)	83
P3	112	B5 (388)	276
P4	426	Not Allocated	-

Best Fit has the best memory utilization because it successfully allocated all four processes (1167 KB used) by saving the largest block for P4. First Fit and Worst Fit failed to allocate P4.

Q10: Page Replacement Algorithms

Page reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 (3 frames).

Apply FIFO, LRU, and Optimal. Calculate total page faults and explain which performs best.

Answer:

FIFO:

7: [7, -, -] (Fault 1)
0: [7, 0, -] (Fault 2)
1: [7, 0, 1] (Fault 3)
2: [2, 0, 1] (Fault 4 - replaces 7)
0: [2, 0, 1] (Hit)
3: [2, 3, 1] (Fault 5 - replaces 0)
0: [2, 3, 0] (Fault 6 - replaces 1)
4: [4, 3, 0] (Fault 7 - replaces 2)
2: [4, 2, 0] (Fault 8 - replaces 3)
3: [4, 2, 3] (Fault 9 - replaces 0)
0: [0, 2, 3] (Fault 10 - replaces 4)
3: [0, 2, 3] (Hit)
2: [0, 2, 3] (Hit)

Total Page Faults for FIFO = 10 page faults.

LRU, replaces the page that has not been used for the longest time:
7: [7, -, -] (Fault 1)
0: [7, 0, -] (Fault 2)
1: [7, 0, 1] (Fault 3)
2: [2, 0, 1] (Fault 4 - replaces 7)
0: [2, 0, 1] (Hit)
3: [2, 0, 3] (Fault 5 - replaces 1 because 0 was just used)
0: [2, 0, 3] (Hit)
4: [4, 0, 3] (Fault 6 - replaces 2)
2: [4, 0, 2] (Fault 7 - replaces 3)
3: [4, 3, 2] (Fault 8 - replaces 0)
0: [0, 3, 2] (Fault 9 - replaces 4)
3: [0, 3, 2] (Hit)
2: [0, 3, 2] (Hit)

Total Page Faults for LRU = 9 page faults.

Optimal:
7: [7, -, -] (Fault 1)
0: [7, 0, -] (Fault 2)
1: [7, 0, 1] (Fault 3)
2: [2, 0, 1] (Fault 4 - replaces 7 because 7 is not used again)
0: [2, 0, 1] (Hit)
3: [2, 0, 3] (Fault 5 - replaces 1 because 1 is not used again)
0: [2, 0, 3] (Hit)
4: [2, 4, 3] (Fault 6 - replaces 0 because 2 and 3 are used sooner than 0)
2: [2, 4, 3] (Hit)
3: [2, 4, 3] (Hit)
0: [2, 0, 3] (Fault 7 - replaces 4 because 4 is not used again)
3: [2, 0, 3] (Hit)
2: [2, 0, 3] (Hit)

Total Page Faults for Optimal = 7 page faults.

Optimal Page Replacement performs best for this reference string because it has the lowest number of page faults (**7 page faults**), compared to **LRU (9 page faults)** and **FIFO (10 page faults)**. Optimal is the best because it looks ahead in the reference string and keeps pages that will be needed soon